

# Rapport technique

Export d'une histoire au format PDF, StoryProject

Djeneba Ba, EFREI Paris, BDML

## 1. Présentation de la fonctionnalité

Cette fonctionnalité permet d'exporter une histoire au format PDF depuis l'écran de détail d'une histoire. Un clic sur le bouton "Exporter en PDF" récupère l'histoire actuellement affichée, ne garde que les scènes dont le statut est Publié, puis génère un document PDF contenant le titre de l'histoire et, pour chaque scène retenue, son titre, son lieu, son moment, la liste de ses personnages présents et son contenu. L'utilisateur choisit lui-même l'emplacement d'enregistrement du fichier sur sa machine, via une boîte de dialogue standard.

Deux ajouts ont été nécessaires pour que cette fonctionnalité ait un sens dans l'application existante. D'abord, un nouveau statut de scène, Publié, qui n'existait pas encore dans l'énumération des statuts. Ensuite, un champ Moment (matin, après-midi, soir...) pour chaque scène, qui n'était jusque-là pas stocké en base, alors que le sujet demande de l'afficher dans le PDF.

## 2. Captures d'écran

Modifier l'histoire

← Accueil | Modifier l'histoire | Exporter en PDF | Supprimer

TITRE  
Le Secret de la Bibliothèque

AUTEUR  
Djeneba Ba

GENRE  
Thriller

RÉSUMÉ 180 caractères  
Une ancienne bibliothèque renferme un secret oublié depuis plusieurs décennies. Plusieurs personnages vont progressivement découvrir des indices menant à une révélation inattendue.

PERSONNAGES + Ajouter  
Emma Etudiante (ProtagonistePrincipal)  
Lucas Journaliste (ProtagonisteSecondaire)  
Victor Leclerc (ProtagonisteSecondaire)  
Madame Moreau (ProtagonisteSecondaire)  
Irla Valentin (Fleurant)

SCÈNES + Ajouter

STATISTIQUES  
Personnages : 5  
Scènes : 5  
Brouillon : 2 En cours : 1 Terminé : 0

Rechercher un mot-clé... Tous les stat... Tous les personn... Réinitialiser

1. La découverte du livre [Brouillon]  
2. Recherche dans les archives [En cours]  
3. Le témoignage de Victor [Publié]  
4. La révélation du secret [Publié]  
10. Le test [Brouillon]

Annuler Sauvegarder

Écran de détail d'une histoire, avec le bouton "Exporter en PDF" ajouté dans l'en-tête, et le statut Publié appliqué à deux scènes en vue de leur export.

# Le Secret de la Bibliothèque

Auteur : Djeneba Ba

## 3. Le témoignage de Victor

Lieu : Université      Moment : Matin

Personnages présents : Lucas Journaliste, Victor Leclerc

En fin de journée, Victor apporte de nouvelles informations concernant l'histoire de la bibliothèque. Le ciel était bleu clair d'un rose teinté

## 4. La révélation du secret

Lieu : Sous-sol de la bibliothèque      Moment : Après-midi

Personnages présents : Emma Etudiante, Lucas Journaliste, Madame Moreau, Victor Leclerc

Le soir, les personnages découvrent enfin le secret caché derrière les archives.

Extrait du document PDF généré : titre de l'histoire, auteur, puis pour chaque scène publiée son lieu, son moment, ses personnages présents et son contenu.

## 3. Classes créées ou modifiées

### PdfExportService (nouvelle classe, couche service)

Classe entièrement nouvelle, responsable uniquement de la mise en page du PDF avec Apache PDFBox. Elle ne fait aucun accès à la base de données ni à l'interface : elle reçoit en paramètres l'histoire, la liste des scènes déjà filtrées, et une correspondance identifiant de personnage vers nom affichable, puis écrit le fichier. Cette séparation permet de l'appeler aussi bien depuis le contrôleur que, plus tard, depuis un test automatisé, sans dépendre de JavaFX.

| Méthode                                                          | Rôle                                                                                                                                                                          |
|------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| exportStory(story, scenesPubliees, nomsPersonnages, destination) | Point d'entrée public. Ouvre le document PDF, écrit le titre et l'auteur de l'histoire, parcourt les scènes publiées, puis enregistre le fichier à l'emplacement demandé.     |
| ecrireScene(scene, nomsPersonnages)                              | Écrit le bloc d'une scène : titre, lieu, moment, personnages présents et contenu, avec un retour à la ligne automatique si le texte est trop long pour la largeur de la page. |
| construirePersonnages(scene, nomsPersonnages)                    | Transforme la liste d'identifiants de personnages liés à une scène en une chaîne de noms lisibles, séparés par des virgules.                                                  |
| nouvellePage() / assurerPlace(hauteur)                           | Gèrent la pagination : une nouvelle page PDF est créée automatiquement dès qu'il ne reste plus assez de place en bas de la page courante.                                     |
| decouperEnLignes(texte, police, taille)                          | Découpe un texte en plusieurs lignes selon la largeur réelle des caractères dans la police choisie, pour que le contenu d'une scène ne déborde jamais de la page.             |

### StatutScene (modifiée, couche modèle)

Ajout d'une nouvelle valeur PUBLIE ("Publié") à l'énumération existante, qui ne comportait jusque-là que Brouillon, En cours et Terminé. C'est ce statut qui sert de critère de sélection des scènes à exporter.

## Scene (modifiée, couche modèle)

Ajout d'un champ moment, avec ses accesseurs getMoment() et setMoment(String), pour stocker le moment de la journée où se déroule la scène. Ce champ était demandé par le sujet mais n'existait dans aucune colonne de la base auparavant.

## SceneRepository (modifiée, couche repository)

Les requêtes SQL d'insertion et de mise à jour, ainsi que la conversion d'une ligne de résultat en objet Scene, ont été mises à jour pour lire et écrire la nouvelle colonne moment.

| Méthode          | Rôle                                                                                        |
|------------------|---------------------------------------------------------------------------------------------|
| save(scene)      | Insère désormais aussi la valeur du champ moment dans la requête INSERT.                    |
| update(scene)    | Met à jour désormais aussi la colonne moment dans la requête UPDATE.                        |
| mapResultSet(rs) | Lit désormais aussi la colonne moment lors de la conversion d'une ligne SQL en objet Scene. |

## StoryController (modifiée, couche contrôleur)

Ajout du champ lié au bouton "Exporter en PDF" et de la méthode qui gère tout le déclenchement de l'export, y compris le passage en traitement d'arrière-plan (voir partie 4).

| Méthode          | Rôle                                                                                                                                                                                                                                             |
|------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| onBtnExportPdf() | Récupère les scènes publiées de l'histoire courante via SceneFilterService, ouvre une boîte de dialogue d'enregistrement, construit la correspondance identifiant vers nom de personnage, puis lance la génération du PDF dans un thread séparé. |

## SceneController (modifiée, couche contrôleur)

Le formulaire de scène lit et enregistre désormais aussi le champ moment, de la même façon que le lieu.

## story-form.fxml et scene-form.fxml (modifiés, couche vue)

Ajout du bouton "Exporter en PDF" dans l'en-tête de l'écran d'une histoire (visible uniquement lorsqu'une histoire est enregistrée, comme le bouton Supprimer), et ajout d'un champ de saisie Moment dans le formulaire de scène, à côté du champ Lieu.

## Base de données MySQL (modifiée)

La table scene existante ne contenait ni la colonne moment, ni la valeur PUBLIE dans l'énumération du statut. Le script suivant a été exécuté dans MySQL Workbench, sur la base storyforge déjà en place, pour ajouter ces deux éléments sans perdre les données existantes :

```
USE storyforge; ALTER TABLE scene ADD COLUMN moment VARCHAR(50) AFTER lieu;
ALTER TABLE scene MODIFY COLUMN statut
ENUM( 'BROUILLON', 'EN_COURS', 'TERMINE', 'PUBLIE' ) NOT NULL DEFAULT 'BROUILLON';
```

Les scènes déjà présentes en base ont conservé leur statut d'origine et une valeur nulle dans la nouvelle colonne moment, à renseigner manuellement pour chacune si besoin.

## pom.xml (modifié)

Ajout de la dépendance Apache PDFBox (org.apache.pdfbox:pdfbox, version 2.0.29), qui fournit les classes PDDocument, PDPage et PDPageContentStream utilisées pour générer le document.

## 4. Exécution en arrière-plan

La génération d'un PDF, même simple, peut prendre un temps non négligeable si l'histoire contient beaucoup de scènes. Si ce travail s'exécutait directement dans `onBtnExportPdf()`, il s'exécuterait sur le thread d'application JavaFX, celui qui gère aussi l'affichage et les clics. L'interface se figerait le temps de l'export, sans possibilité de l'annuler ni même de voir qu'elle répond encore.

La solution retenue utilise `javafx.concurrent.Task`, la classe prévue par JavaFX pour ce cas d'usage. Le travail réel (l'appel à `PdfExportService.exportStory()`) est placé dans la méthode `call()` d'un `Task<Void>`. Ce Task est ensuite exécuté dans un Thread séparé, créé et démarré explicitement (`thread.setDaemon(true)` pour qu'il ne bloque pas la fermeture de l'application, `thread.start()` pour le lancer).

Le Task expose trois événements auxquels le contrôleur réagit, et qui sont automatiquement ramenés sur le thread JavaFX par le Task lui-même, donc sans risque de modifier l'interface depuis le mauvais thread : `setOnRunning()` désactive le bouton et change son texte pendant l'export, `setOnSucceeded()` réaffiche le bouton et montre une confirmation avec le chemin du fichier créé, et `setOnFailed()` réaffiche le bouton et montre le message d'erreur si l'écriture du fichier a échoué. L'utilisateur peut donc continuer à naviguer dans l'application pendant que le fichier s'écrit sur le disque.

## Sources

Documentation officielle Apache PDFBox : <https://pdfbox.apache.org/>

Documentation officielle JavaFX, Concurrency (Task, Thread) :  
<https://openjfx.io/javadoc/21/javafx.graphics/javafx/concurrent/Task.html>